



Università di Genova

High Performance Computing Report Mandelbrot Algorithm

Andrea Cattarinich 5137057

November 28, 2025

Contents

1	Architectures	2
1.1	Personal Workstation	2
1.2	University Workstation	2
1.3	Google Colab	3
2	Mandelbrot Set	4
2.1	Definition	4
2.1.1	Point that belongs to the Mandelbrot set	4
2.1.2	Point that does NOT belong to the Mandelbrot set	4
2.2	Images	4
3	Hotspot Analysis	5
3.1	Code Description	5
3.2	Project Structure	6
3.3	Sequential and Parallel Sections	7
3.4	Compilation	7
3.5	Hotspot Identification	8
3.6	Conclusions	8
4	Conclusions	10

1 Architectures

For this project I have used three workstation, my personal one, the workstation provided by the University and Google Colab.

1.1 Personal Workstation

HP Envy 17-ae103nl

CPU

Model	Intel Core i7-6500U @ 2.50 GHz
Architecture	x86_64
Physical cores	2
Logical threads	4 (Hyper-Threading)

GPU

Model	NVIDIA GeForce GTX 950M
CUDA Compute Capability	5.0
Driver version	572.16
CUDA Version	12.8

Software and Setup

Visual Studio	Visual Studio Community 2022 (C++ Desktop workload)
CUDA Toolkit	Installed manually
WSL2	Enabled
NVIDIA Drivers	Version 572.16 (for WSL2)

1.2 University Workstation

Room 210

CPU

Model	Intel Core i9-12900K (12th Gen)
Architecture	x86_64
CPU(s)	24
Threads per core	2
Cores per socket	16
Socket(s)	1
Max frequency	4.02 GHz
Profiling tools	Intel VTune Profiler, Intel Advisor

To use the Intel oneAPI tools, the environment was loaded with:

```
source /opt/intel/oneapi/setvars.sh
```

1.3 Google Colab

CPU

Model	Intel(R) Xeon(R) CPU @ 2.20GHz
Architecture	x86_64
Physical cores	1
Logical threads	2 (Hyper-Threading)
Socket(s)	1

GPU

Model	NVIDIA Tesla T4
CUDA Compute Capability	7.5
Driver version	550.54.15
CUDA Version	12.8

2 Mandelbrot Set

2.1 Definition

The Mandelbrot fractal is defined as a set of points in the complex plane. Each point corresponds to a complex number c .

For every c , we consider the iterative sequence:

$$z_0 = 0, \quad z_{n+1} = z_n^2 + c$$

We then observe the behavior of z_n as $n \rightarrow \infty$.

2.1.1 Point that belongs to the Mandelbrot set

Take $c = -1$. Then the sequence is

$$z_0 = 0, \quad z_1 = 0^2 + (-1) = -1, \quad z_2 = (-1)^2 + (-1) = 0, \quad z_3 = 0^2 + (-1) = -1, \dots$$

The results remain bounded, so $c = -1$ belongs to the Mandelbrot set.

2.1.2 Point that does NOT belong to the Mandelbrot set

Take $c = 1$. Then the sequence is

$$z_0 = 0, \quad z_1 = 0^2 + 1 = 1, \quad z_2 = 1^2 + 1 = 2, \quad z_3 = 2^2 + 1 = 5, \dots$$

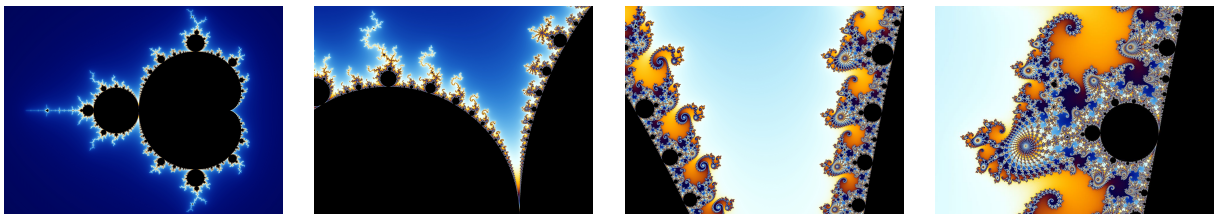
The values grow up to infinite, so $c = 1$ does not belong to the Mandelbrot set.

In our specific case, the escape condition is defined as:

$$|z_i| > 2$$

2.2 Images

This process gives rise to the famously intricate structure of the Mandelbrot fractal. Here a simple explanation [1] or here a more complex one [2] and here an online demo [3].



The color represents how quickly the values of z_n grow:

Black - the sequence remains bounded;

Other colors - the sequence escapes to infinity, with the color indicating how fast it diverges.

3 Hotspot Analysis

The Fourier Transform is a fundamental mathematical tool in digital signal processing. It converts a sequence of values from the **time domain** into a sequence of values in the **frequency domain**. This transformation makes it possible to identify which frequencies are present in a signal and with what intensity.

The continuous-time Fourier Transform is defined as:

$$X(\omega) = \int_{-\infty}^{+\infty} x(t) e^{-j\omega t} dt$$

The example shown in Figure 1 illustrates the practical and optical interpretation of the Fourier Transform of a rectangular signal, showing how a simple time-domain shape corresponds to a characteristic frequency-domain spectrum.

Source		Assembly			
Source Line ▲	Source	🔥 CPU Time: Total ⓘ	CPU Time: Self ⓘ		
31					
32	const auto start = chrono::steady_clock::now();				
33	for (int pos = 0; pos < HEIGHT * WIDTH; pos++)	0.0%	0.008s		
34	{				
35	image[pos] = 0;	0.0%	0.012s		
36					
37	const int row = pos / WIDTH;	0.0%	0.008s		
38	const int col = pos % WIDTH;	0.0%	0.012s		
39	const complex<double> c(col * STEP + MIN_X, row * STEP + MIN_Y);	0.0%	0.012s		
40					
41	// z = z^2 + c				
42	complex<double> z(0, 0);				
43	for (int i = 1; i <= ITERATIONS; i++)				
44	{				
45	z = pow(z, 2) + c;	39.9%	3.570s		
46					
47	// If it is convergent				
48	if (abs(z) >= 2)	59.9%	0.732s		
49	{				
50	image[pos] = i;				
51	break;				
52	}				
53	}	0.0%	0.024s		
54	}	0.0%	0.012s		

Figure 1: Fourier Transform of a rectangular signal.

3.1 Code Description

The algorithm provided by the course implements both the Discrete Fourier Transform (DFT) and its inverse (IDFT) in C. It computes the transform of a signal and verifies the correctness by applying the IDFT, following the discrete-time Fourier Transform definition:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}, \quad k = 0, 1, \dots, N-1$$

The DFT is implemented following the standard quadratic-complexity $O(N^2)$ formulation, with two nested loops over the frequency index k and the time index n . Each output element can be computed independently, which makes the algorithm naturally parallelisable.

```

1 int main() {
2     int N = 100000;
3     double *xr, *xi, *Xr, *Xi;
4
5     fillInput(xr, xi, N);           // initialize input signal
6     zero(Xr, Xi, N);               // clear output buffers
7
8     DFT(+1, xr, xi, Xr, Xi, N);     // compute DFT
9     DFT(-1, Xr, Xi, xr, xi, N);     // compute IDFT
10
11     checkResults(xr, xi, Xr, Xi, N); // verify correctness
12
13     return 0;
14 }
15
16 void DFT(int idft, double* xr, ..., double* Xi, int N) {
17     for (int k = 0; k < N; k++)
18         for (int n = 0; n < N; n++) {
19             double a = 2*M_PI*n*k/N;
20             Xr[k] += xr[n]*cos(a) + idft*xi[n]*sin(a);
21             Xi[k] += -idft*xr[n]*sin(a) + xi[n]*cos(a);
22         }
23
24     if (idft == -1)
25         for (int n = 0; n < N; n++) {
26             Xr[n] /= N;
27             Xi[n] /= N;
28         }
29 }

```

Listing 1: Pseudo code in C

3.2 Project Structure

The `homework1` project is organized into four main directories:

- **build**: contains the compiled executables.
- **reports**: stores the reports generated with the `-qopt-report` flag.
- **src**: includes the source code files.
- **metrics**: scripts that run the executables in **build** and save performance metrics.

The first step to improving the performance of a program is profiling. The goal of this analysis is to identify the parts of the program that consume the most execution time. To achieve this, I considered a data size resulting in a sequential execution time of ~ 30 seconds, by setting $N = 60'000$.

From this point onward, each executable is empirically measured over 10 runs to obtain reliable results.

3.3 Sequential and Parallel Sections

In order to apply Amdahl's Law (discussed in more detail in the Chapter ??), we must identify the sequential and parallel parts of the code. I have empirically measured these sections by separating the serial and parallel portions. More details of how I obtained these results can be found in the code `sequential_parallel_code_identification.c`.

The results are the following:

Quantity	Value
Total execution time T_{tot}	32.48231 s
Time of the parallelizable section T_{par}	32.48021 s
Time of the serial section $T_{\text{ser}} = T_{\text{tot}} - T_{\text{par}}$	0.00209 s
Fraction of serial code $f_s = \frac{T_{\text{ser}}}{T_{\text{tot}}}$	0.006 %

These values will be used to estimate the theoretical speedup according to Amdahl's Law:

$$S(p) = \frac{1}{f_s + \frac{1-f_s}{p}} \Big|_{f_s=0.00006} = \frac{1}{0.00006 + \frac{1-0.00006}{p}} \approx p$$

Note: the smaller the serial fraction f_s , the higher the potential speedup achievable by parallelizing the main loop of the DFT.

Note: in the real case, the speedup is not linear. In particular, from the parallelization examples presented in Chapter ??, the speedup is approximately linear up to 8 threads; beyond this point, it deviates from linear behavior.

3.4 Compilation

Since the Intel(R) C++ Compiler Classic (ICC) is deprecated and was removed from product release in the second half of 2023. The Intel(R) oneAPI DPC++/C++ Compiler (ICX) was used to performe the analysis.

To identify the optimal sequential compilation strategy, I conduced testing across multiple Intel compiler variants (ICC and ICX), focusing on the vectorization report provided by the flag `-qopt-report`.

The program was compiled using the Intel compiler ICX with the following command (more details in Chapter ??):

```
icx -O2 -vec -g \
    -fopenmp \
    -qopt-report=max \
    -qopt-report-file=reports/omp_homework.optrpt \
    -o build/omp_homework \
    src/omp_homework.c
```

The compilation flags have the following meaning:

- `-O2`: optimizations for better performance;

- `-fopenmp`: activates compilation for OpenMP directives;
- `-vec`: activates automatic vectorization of loops using SIMD instructions;
- `-g`: includes debugging information in the executable;
- `-qopt-report=max`: generates a detailed optimization report;
- `-qopt-report-file=...`: specifies the file where the optimization report is saved;
- `-o build/omp_homework`: sets the name and location of the compiled executable;
- `src/omp_homework.c`: specifies the source file to compile.

3.5 Hotspot Identification

The measurements were performed using Intel VTune Profiler.

Function Stack	Total	Percentage (%)
Total	69.97	100.00
_start	69.97	100.00
__libc_start_main_impl	69.97	100.00
– main	69.97	100.00
— std::abs<double>	41.2121	58.90
— std::operator+<double>	13.6361	19.49
— std::pow<double>	10.712	15.31
— std::complex<double>::complex	0.0199997	0.03

Source Line ▲	Source	CPU Time: Total	CPU Time: Self
31			
32	const auto start = chrono::steady_clock::now();		
33	for (int pos = 0; pos < HEIGHT * WIDTH; pos++)	0.0%	0.008s
34	{		
35	image[pos] = 0;	0.0%	0.012s
36			
37	const int row = pos / WIDTH;	0.0%	0.008s
38	const int col = pos % WIDTH;	0.0%	0.012s
39	const complex<double> c(col * STEP + MIN_X, row * STEP + MIN_Y);	0.0%	0.012s
40			
41	// z = z^2 + c		
42	complex<double> z(0, 0);		
43	for (int i = 1; i <= ITERATIONS; i++)		
44	{		
45	z = pow(z, 2) + c;	39.9%	3.570s
46			
47	// If it is convergent		
48	if (abs(z) >= 2)	59.9%	0.732s
49	{		
50	image[pos] = i;		
51	break;		
52	}		
53	}	0.0%	0.024s
54	}	0.0%	0.012s

3.6 Conclusions

The main hotspot is the `DFT()` function and its inverse `IDFT()`, which together account for over 99,05% of the total execution time. All other functions have negligible impact. This indicates that **parallelization and vectorization efforts should focus on the double loop inside `DFT()`**.

The main hotspot is the loop contained in lines 72-80 (source: src/omp_homework.c). The dominant operations contributing to the algorithm's computational intensity are the `sin()` and `cos()` trigonometric functions, which account for the majority of the execution time, as shown in the following code.

Source		Assembly			
Source Line ▲	Source	🔥 CPU Time: Total ▾	CPU Time: Self ▾		
31					
32	const auto start = chrono::steady_clock::now();				
33	for (int pos = 0; pos < HEIGHT * WIDTH; pos++)	0.0%	0.008s		
34	{				
35	image[pos] = 0;	0.0%	0.012s		
36					
37	const int row = pos / WIDTH;	0.0%	0.008s		
38	const int col = pos % WIDTH;	0.0%	0.012s		
39	const complex<double> c(col * STEP + MIN_X, row * STEP + MIN_Y);	0.0%	0.012s		
40					
41	// z = z^2 + c				
42	complex<double> z(0, 0);				
43	for (int i = 1; i <= ITERATIONS; i++)				
44	{				
45	z = pow(z, 2) + c;	39.9%	3.570s		
46					
47	// If it is convergent				
48	if (abs(z) >= 2)	59.9%	0.732s		
49	{				
50	image[pos] = i;				
51	break;				
52	}				
53	}	0.0%	0.024s		
54	}	0.0%	0.012s		

4 Conclusions

References

- [1] Simple Explanation of the Mandelbrot Set.
<https://www.wikihow.com/Plot-the-Mandelbrot-Set-By-Hand>
- [2] Mandelbrot Set – Formal Explanation.
https://en.wikipedia.org/wiki/Mandelbrot_set
- [3] Online Mandelbrot Demo.
<https://math.hws.edu/eck/js/mandelbrot/MB.html>